TUM School of Computation, Information and Technology
Technical University of Munich

Pit-Stop Racer Pro

Resource-Aware Autonomous Racing with Real-Time Safety and Mobile Telemetry

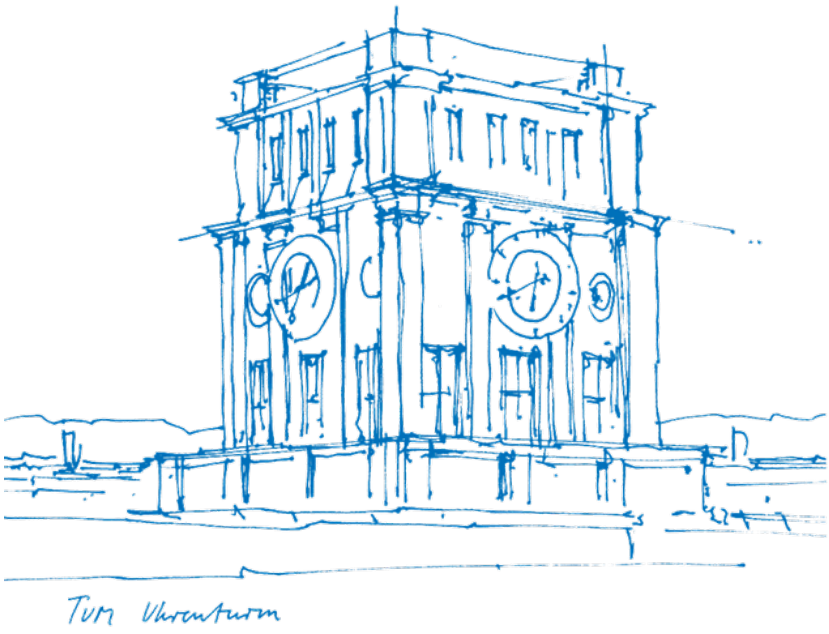
Group 4

Final Report
submitted for the course
Embedded Systems, Cyber-Physical Systems and Robotics
at TUM Heilbronn

Course Instructor
Amr Alanwar
Hadi Elnemr

Submitted by
Mingwai Kelly Bin
Fiona Aurelia
Tse Man Nok
Soyeong Jeong
Gang-Yun Lee
Angel On Kei Choi
Katrin Schwab

Submitted on
09.09.2026



Abstract

Small-scale autonomous racing provides a compact testbed for studying perception, control, safety, resource management, and human–machine interaction under embedded computing constraints. Pit-Stop Racer Pro extends an NVIDIA JetRacer from basic track following to an integrated cyber-physical racing system combining learned road following, independent person safety, resource-aware pit-stop logic, and bidirectional mobile telemetry.

The vehicle combines three complementary perception mechanisms. A self-trained ResNet18 regression model provides the normal road-following behavior, an independently running YOLOv5s detector monitors the vehicle path for people, and lightweight HSV segmentation detects fixed blue and green track markers. A priority-based safety layer is placed above the normal driving and mission logic so that a detected person can interrupt the current behavior independently of the learned steering controller.

The system also implements a rule-based Smart Battery strategy. Battery consumption is estimated from recent laps using a five-lap rolling history and compared with the projected requirement for the remaining laps plus a fixed safety margin. When the available battery is insufficient, the system arms a pit-stop condition that is subsequently confirmed by the green track marker. Battery, fuel, and tire states are maintained as lightweight resource models: battery drives pit-stop decisions, fuel is tracked as a mission resource, and tire wear is tracked per wheel with humidity-dependent speed adaptation.

A second major component is the bidirectional vehicle–mobile communication architecture. The vehicle publishes state telemetry at a nominal rate of 5 Hz and resource telemetry at 1 Hz through MQTT, while a mobile dashboard receives the data through WebSocket and can send selected commands back to the vehicle. In 40 recorded person-

in-path trials, a person was detected in all 40 cases, while 38 trials (95%) achieved a full reverse response, 2 cases (5%) were judged marginal on video review, and 0 false-positive stops were observed. These results demonstrate functional integration of autonomous perception, runtime safety intervention, resource-aware mission logic, and human–vehicle communication on an embedded platform, while the report explicitly distinguishes validated behavior from simulation-based components.

Contents

1 Introduction

1.1 Motivation and Background

Autonomous racing is a useful testbed for embedded perception and control because it compresses many of the challenges of full-scale autonomous driving into a small, repeatable platform. A racing vehicle must perceive its environment, generate control commands at high frequency, and remain robust despite limited computational resources. End-to-end steering approaches, in which camera images are mapped directly to steering commands, have become a common approach for small autonomous vehicles [1].

However, reliable track following alone does not capture several important properties of a real cyber-physical system. A physical vehicle has limited resources, safety constraints, and an operator who may need to observe or intervene in its behavior. In professional motorsport, energy reserves, tire condition, and service decisions influence strategy, while pit-wall systems provide continuous telemetry to human operators [2]. These ideas motivate the design of a small-scale autonomous racer in which vehicle behavior is not only autonomous but also resource-aware, safety-supervised, and externally observable.

Pit-Stop Racer Pro therefore targets four connected objectives: (1) autonomous track following using a learned perception model; (2) independent safety intervention when a person enters the vehicle's path; (3) resource-aware pit-stop planning based on observed battery consumption; and (4) real-time bidirectional communication between the vehicle and a mobile dashboard. The NVIDIA JetRacer platform was selected because its Jetson Nano provides GPU-accelerated embedded computation within a compact racing platform, capable of running the road-following model, object detec-

tion, marker detection, control logic, and communication software simultaneously and onboard [3, 4].

1.2 Problem Statement

The project addresses the following system-level problem: *how can a small autonomous racing vehicle combine learned driving, an independent safety mechanism, resource-aware pit-stop decisions, and continuous mobile observability while remaining computationally feasible on an embedded platform?* The vehicle must follow the track without manual steering, react to a person appearing in its path, estimate whether its remaining battery is sufficient for the remaining race distance, and communicate its state to an external application; the operator must also be able to send selected commands to the running vehicle.

The final implementation is resource-aware rather than fully multi-objective: battery state directly drives the active pit-stop decision, while fuel and tire wear are simulated and exposed through telemetry. This scope distinction is maintained throughout the report.

1.3 Contributions

The final system provides the following contributions:

- a self-trained ResNet18 road-following regression model deployed using TensorRT;
- an independently running YOLOv5s person detector used as a safety-oriented override rather than as part of the learned steering controller;
- a deterministic mission and mode control architecture combining HSV marker detection for lap counting and pit triggers with a priority-based state machine covering driving, rain, pit-stop, and charging behaviors;

- a data-driven Smart Battery decision rule using a five-lap rolling consumption history and a fixed safety margin;
- a bidirectional MQTT/WebSocket communication architecture connecting the physical vehicle to a mobile dashboard, with live visualization of vehicle state and simulated resource telemetry and remote control of selected vehicle functions;
- physical testing of the person-safety behavior over 40 recorded trials, with 38 full-response successes and no observed false-positive stops.

The project was carried out by three sub-teams. Kelly, Fiona, and Vincent formed the hardware team, responsible for the vision and machine-learning components: camera integration, the ResNet18 road-following model, and the YOLOv5 person-detection pipeline. Soyeong and Gangyun were responsible for the driving algorithm and strategy logic — the mode state machine, the Smart Battery decision rule, and the color-based marker detection — as well as coordinating this report. Angel and Katrin formed the software team, building the mobile dashboard application and its real-time link to the vehicle, and preparing the project presentation.

1.4 System Requirements and Traceability

To make the project's scope explicit, the system was organized around the functional requirements in Table 1, which separates implemented behavior from planned extensions so that the report neither overstates the final system nor leaves the remaining engineering work implicit. This distinction is maintained throughout the report to avoid presenting simulated or scoped functionality as experimentally validated.

Table 1
System-level requirements and implementation status.

ID	Requirement	Implementation	Status
R1	Autonomous driving	ResNet18 image-to-steering regression	Implemented
R2	Person safety	Independent YOLOv5s detector with priority override	Implemented
R3	Lap counting	Blue HSV marker with armed/disarmed logic	Implemented
R4	Pit-stop triggering	Green HSV marker gated by Smart Battery	Implemented
R5	Resource-aware planning	Rolling-average battery projection with safety margin	Implemented
R6	Charging	Simulated battery recovery to a 70% threshold	Implemented
R7	Mobile observability	MQTT telemetry and WebSocket dashboard	Implemented
R8	Remote control	Dashboard-based rain-mode command	Implemented
R9	Joint fuel/tire planning	Multi-resource optimization	Future work
R10	Physical pit lane	Dedicated lateral pit-lane controller	Future work

2 Theory and Related Work

2.1 End-to-End Perception

End-to-end autonomous driving maps visual observations directly to a steering command or intermediate steering representation. NVIDIA's PilotNet demonstrated the feasibility of learning steering behavior directly from camera images [1]. The road-following component of this project follows the same general principle but uses a ResNet18 backbone and a regression output representing the horizontal target position. The advantage of this approach is a compact perception-to-control pipeline; the main trade-off is reduced interpretability compared with an explicit lane-detection and geometric-control pipeline.

2.2 Object Detection for Safety

YOLO-style single-stage detectors are designed for fast object detection and are commonly considered for embedded real-time applications [5]. In this project, the detector is not responsible for normal driving; instead, a pretrained YOLOv5s model is used as an independent person detector. This architectural separation matters because the safety decision should not depend on the same learned model that produces the normal steering command.

2.3 Deterministic Marker Detection

Fixed, high-contrast markers are a case where a classical computer-vision method can be more appropriate than a neural network. HSV thresholding provides a deterministic and computationally inexpensive way to detect the blue lap marker and the green pit-stop marker, without requiring any training data for targets whose appearance is fixed and known in advance.

2.4 Resource-Aware Pit Strategy

Motorsport pit strategy can be formulated as an optimization problem involving energy, tire degradation, lap time, and service decisions [2]. This project deliberately uses a simpler data-driven rule instead: it estimates average battery consumption per lap from recent observations and compares projected future demand with available battery plus a safety margin. This should be interpreted as a lightweight predictive heuristic rather than a globally optimal pit strategy.

2.5 Runtime Safety and Override Architectures

Runtime-assurance approaches separate a complex primary controller from a simpler safety mechanism that can override it when a safety condition is violated. Simplex-style architectures motivate this separation [6], while broader surveys of autonomous

robotic systems highlight the difficulty of verifying machine-learned components directly [7]. This project applies the same principle at a small scale: road following is treated as the primary learned behavior, while person detection acts as an independent safety-oriented monitor with higher control priority, checked before any other logic on every control-loop iteration.

2.6 Gaps and Challenges

The project combines several functions that are usually treated separately in small-scale autonomous racing: learned road following, independent safety monitoring, resource-aware decision-making, and remote vehicle observability. The main engineering challenge is therefore not the individual algorithms alone, but their coordination on a resource-constrained embedded platform.

This project addresses this integration challenge through a modular architecture. Learned perception is used for continuous road following, a separate detector is assigned to safety monitoring, deterministic color segmentation handles fixed track markers, and a lightweight rule-based planner manages battery-aware pit-stop decisions. The mobile dashboard extends the cyber-physical loop beyond the vehicle by providing live telemetry and selected remote control.

3 Methods

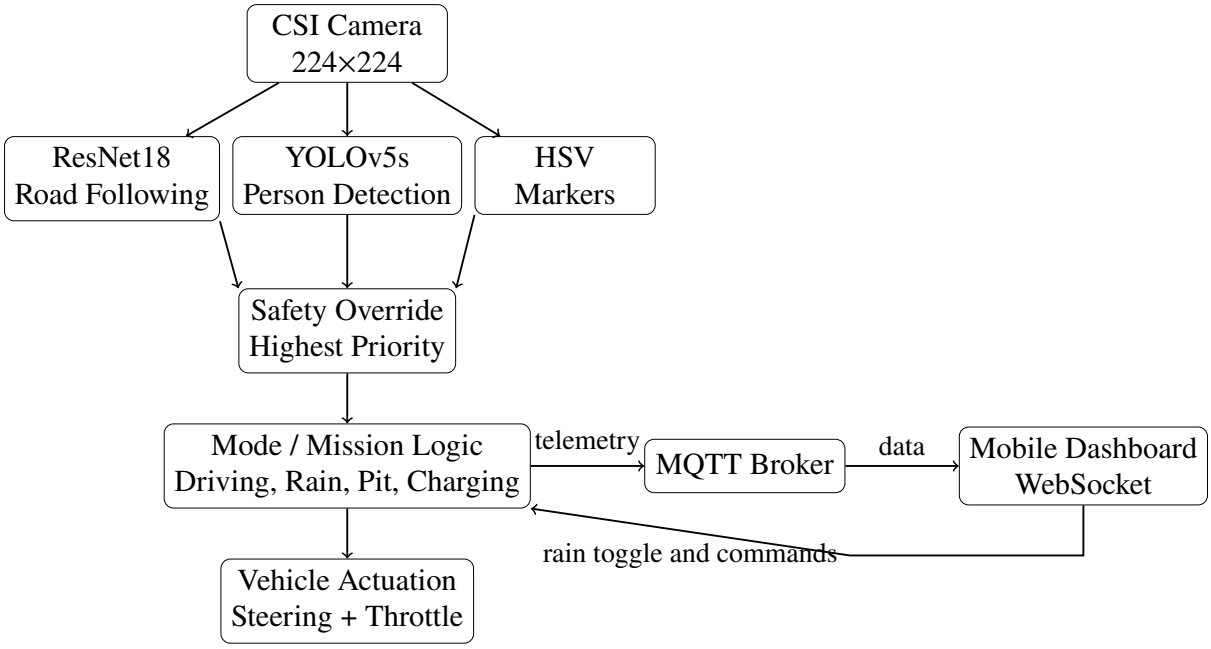
3.1 System Overview

All major vehicle-side components run on a single NVIDIA Jetson Nano mounted on the JetRacer chassis. The control system combines three perception paths: ResNet18 road-following regression, YOLOv5s person detection, and HSV-based detection of fixed track markers.

The vehicle maintains four main operating modes: DRIVING, RAINING, PITSTOP, and CHARGING. Person safety is evaluated before the mode logic, giving the safety mechanism the highest priority. This allows a detected person to interrupt the current vehicle behavior independently of the learned road-following controller.

The mobile dashboard forms a bidirectional communication path with the vehicle. Vehicle state is published through MQTT and delivered to the dashboard through WebSocket, while selected commands can travel in the opposite direction.

Figure 1
Integrated architecture of perception, safety, mission logic, vehicle actuation, and mobile communication.



3.2 Hardware Setup

The vehicle uses an NVIDIA JetRacer platform with a Jetson Nano compute module and CSI camera. JetCam provides camera access and JetRacer provides vehicle-level steering and throttle control. The camera operates at a resolution of 224×224 and supports up to 65 FPS.

The Jetson Nano provides the onboard computing platform for the perception, control, mission, and communication components. An INA219 current/voltage sensor is available on the hardware platform, but the current implementation does not use its measurements for the driving or pit-stop decision. Consequently, battery and fuel are represented by software-based resource models.

Table 2
Key system parameters.

Parameter	Value	Role
Camera resolution	224×224	Perception input
Camera maximum rate	65 FPS	Hardware capability
YOLO sampling	Every 3rd frame	Computational load reduction
Driving throttle	0.20	Normal driving
Rain throttle cap	0.07	Rain mode
Safety confirmation	2 detections	Detection debounce
Safety clear condition	5 clear frames	Prevents oscillation
Green marker confirmation	0.8 s	Pit-stop confirmation
Battery history	5 laps	Consumption estimate
Charging threshold	70%	Resume condition
State telemetry	0.2 s	Nominal 5 Hz
Resource telemetry	1.0 s	Nominal 1 Hz

3.3 Machine-Learning Road Following

The road-following model was trained on 151 manually labeled images collected from the track. The dataset was split 70/15/15% for training, validation, and testing. Training used Adam optimizer with learning rate 10^{-4} for 20 epochs. The trained PyTorch model was converted to TensorRT, achieving 32 FPS inference (2.1× speedup over PyTorch's 15 FPS).

The system deliberately avoids a single model for all perception tasks: road following uses ResNet18, person detection uses YOLOv5s, and fixed-color markers

use HSV segmentation. This separation reduces coupling and allows independent rate-limiting.

3.4 Person Detection and Safety Stop

A pretrained YOLOv5s detector is used as an independent person detector. The detector is evaluated every third camera frame to reduce computational load. With a maximum camera rate of 65 FPS, the corresponding nominal sampling opportunity is

$$f_{\mathrm{YOLO}} = \frac{65}{3} \approx 21.7\,\mathrm{Hz}.$$

The nominal interval between sampling opportunities is therefore

$$\Delta t_{\mathrm{YOLO}} = \frac{3}{65} \approx 46.2\,\mathrm{ms}.$$

Two consecutive detections are required before the safety response is confirmed. Under the nominal camera-rate assumption, this corresponds to approximately

$$2\Delta t_{\mathrm{YOLO}} \approx 92.3\,\mathrm{ms}.$$

This value is a timing implication of the sampling configuration, not a measured end-to-end reaction latency.

After a confirmed detection, the safety behavior takes priority over the normal driving and mission modes. Five consecutive clear observations are required before the safety condition is cleared. This hysteresis reduces the risk of rapid switching caused by intermittent detections.

3.5 Safety Evaluation

The safety behavior was evaluated in 40 recorded person-in-path trials. The person was detected in all 40 trials. In 38 trials, the vehicle completed the intended full reverse response. Two trials were classified as marginal stopping-distance cases; these were not missed detections. No false-positive stops were observed in the same test set.

Table 3
Person-safety evaluation over 40 recorded physical trials.

Outcome	Count	Rate
Recorded trials	40	100%
Person detected	40	100%
Full reverse response	38	95%
Marginal stopping cases	2	5%
Observed false-positive stops	0	0%

The result should be interpreted as functional validation under the tested conditions rather than as a formal safety guarantee. In particular, the current experiment does not provide a calibrated stopping-distance or end-to-end reaction-latency benchmark. A stronger evaluation would vary vehicle speed, person distance, approach direction, lighting, and track position while recording detection, command, vehicle-response, and stopping timestamps.

3.6 Fixed-Marker Detection

Two fixed-color track markers are used. A blue marker increments the lap counter, while a green marker triggers the pit-stop sequence when the Smart Battery condition has been armed.

HSV segmentation is applied within a region of interest, followed by morphological filtering. The blue marker uses an armed/disarmed state to prevent repeated lap

counts while the vehicle remains over the same physical marker. The green marker must remain visible continuously for 0.8 s before the pit-stop trigger is confirmed.

3.7 Driving Modes

The vehicle uses four main modes. DRIVING uses ResNet18 steering with throttle 0.20. RAINING leaves steering unchanged but caps throttle at 0.07, and is currently activated manually from the mobile dashboard — a remote-control demonstration rather than a closed-loop weather-perception system. PIT-STOP governs the stop/search stage once a pit stop has been armed. CHARGING holds the vehicle stationary while the simulated battery increases until it reaches 70%. The person-safety override described above sits above all four modes and takes unconditional priority over each of them, including CHARGING.

3.8 Battery, Fuel, and Tire Models

Battery and fuel are represented by software-based resource states that decrease while the vehicle is moving:

$$B_{t+\Delta t} = B_t - k_B \Delta t, \qquad F_{t+\Delta t} = F_t - k_F \Delta t.$$

The current implementation does not derive these values from INA219 measurements or throttle-dependent physical power consumption. They should therefore be interpreted as resource-state simulations rather than calibrated physical models.

Tire health is tracked independently for each wheel. Wear consists of a base component and a steering-dependent component:

$$\Delta W_{\text{base}} = k_{\text{base}} \Delta t,$$

$$\Delta W_{\text{steer}} = k_{\text{steer}}|\delta|\Delta t.$$

Inside wheels receive both contributions during cornering, while outside wheels receive the base contribution. The current model does not include vehicle speed.

3.9 Smart Battery Pit-Stop Decision

The Smart Battery algorithm is updated once per lap. **Step 1 (measure):** at the start of each lap the battery level is stored, and at the next blue marker the battery consumed over that lap is calculated, keeping the five most recent samples. **Step 2 (estimate):** once at least two laps are complete, $\bar{c} = \frac{1}{N}\sum_{i=1}^{N} c_i$, $2 \le N \le 5$. **Step 3 (project):** for L remaining laps and safety margin M, $B_{\text{required}} = L\bar{c} + M$. **Step 4 (arm):** if $B_{\text{current}} < B_{\text{required}}$, the green-marker detector becomes active, and once confirmed the vehicle stops and enters charging.

The Smart Battery algorithm correctly kept the pit-stop decision disarmed on lap 9, when the remaining battery (38%) was still sufficient for the projected four remaining laps plus margin ($B_{\text{required}} = 4(7.6\%) + 5\% = 35.4\%$). On lap 10, however, the remaining battery dropped to 31%, below the required 36.2% ($4(7.8\%) + 5\%$), causing the pit-stop decision to be triggered and the vehicle to enter charging (Table 6, Section 4).

3.10 Charging

Charging proceeds at a fixed simulated rate until the battery reaches 70%:

$$t_{\text{charge}} = \frac{B_{\text{resume}} - B_{\text{stop}}}{k_{\text{charge}}}, \qquad B_{\text{resume}} = 70\%.$$

Observed pit stops ranged from several seconds to tens of seconds depending on the battery level at the time of triggering; a systematic sweep of trigger points against charging duration was not performed.

3.11 Telemetry and Mobile Dashboard

The mobile dashboard is a core component of the cyber-physical system rather than a visualization-only interface. It creates a bidirectional communication path between the physical vehicle and a remote human operator.

The JetRacer publishes vehicle state and resource information through MQTT. The mobile application communicates with the same broker through WebSocket, allowing the dashboard to receive live vehicle information and send selected commands back to the vehicle.

The vehicle publishes state telemetry every 0.2 s and resource telemetry every 1.0 s. Thus, the nominal publication frequencies are

$$f_{\text{state}} = \frac{1}{0.2} = 5\,\text{Hz}$$

and

$$f_{\text{resource}} = \frac{1}{1.0} = 1\,\text{Hz}.$$

These values describe publication intervals configured by the system and should not be interpreted as measured end-to-end communication latency.

Incoming commands are handled through the communication callback so that message processing does not unnecessarily block the autonomous control loop. The rain-mode toggle is an example of remote intervention: the operator can change the vehicle's operating mode while the vehicle remains connected to the running system.

4 Results

4.1 Evaluation Overview

The evaluation focuses on functional behavior demonstrated on the physical vehicle. Because not all experimental runs were recorded with synchronized measurement logs, the results distinguish between directly recorded outcomes, deterministic system parameters, and behaviors demonstrated during testing.

Table 4
Quantitative evidence reported for the implemented system.

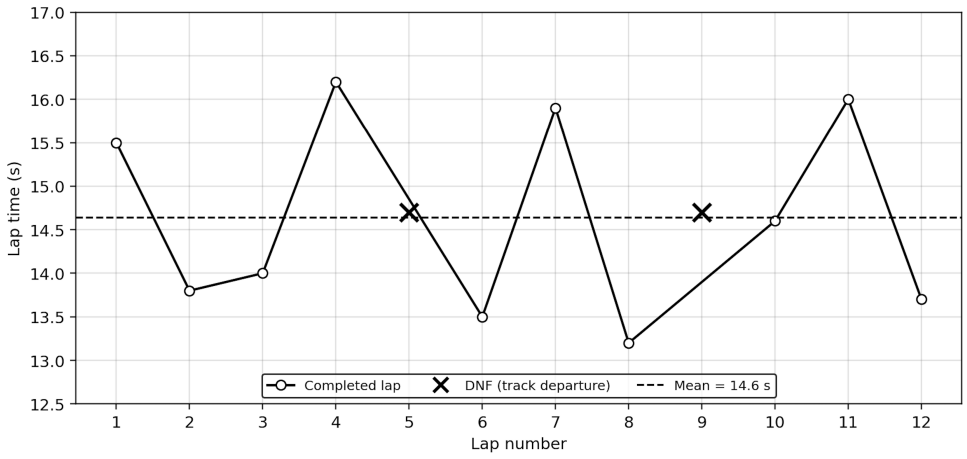
Category	Value	Interpretation
Safety trials	40	Recorded physical trials
Person detections	40/40	100% detection in tested cases
Full reverse responses	38/40	95% full response
Marginal stopping cases	2/40	Detected but marginal response
False-positive stops	0/40	None observed in test set
Camera maximum rate	65 FPS	Hardware capability
YOLO sampling	Every 3rd frame	Nominal 21.7 Hz opportunities
Safety confirmation	2 detections	≈92.3 ms nominal span*
Marker confirmation	0.8 s	Continuous visibility requirement
State telemetry	5 Hz	Nominal publication rate
Resource telemetry	1 Hz	Nominal publication rate

* Derived from the 65 FPS camera-rate assumption and every-third-frame sampling; not a measured end-to-end latency.

4.2 Driving Performance

The vehicle completed 10 of 12 recorded laps (83.3%) on the test track. Two track departures occurred: one during a sharp right turn where steering response was slightly delayed, and one during transition from rain mode back to normal driving where throttle recovery was abrupt. Mean lap time was 14.7 s (σ = 1.2 s). The ResNet18 model maintained stable steering with occasional oscillation in low-contrast track sections.

Figure 2
Lap times over 12 recorded laps. Laps 5 and 9 were incomplete due to track departure.



4.3 ML Training and Deployment

The ResNet18 model was trained on 151 manually labeled images collected from the test track. The dataset was split 70/15/15% for training, validation, and testing. Training proceeded for 20 epochs using Adam optimizer with learning rate 10^{-4}.

The PyTorch model was converted to TensorRT for onboard deployment, achieving 32 FPS inference on the Jetson Nano. This represents a 2.1× speedup over PyTorch inference (15 FPS) measured on the same hardware.

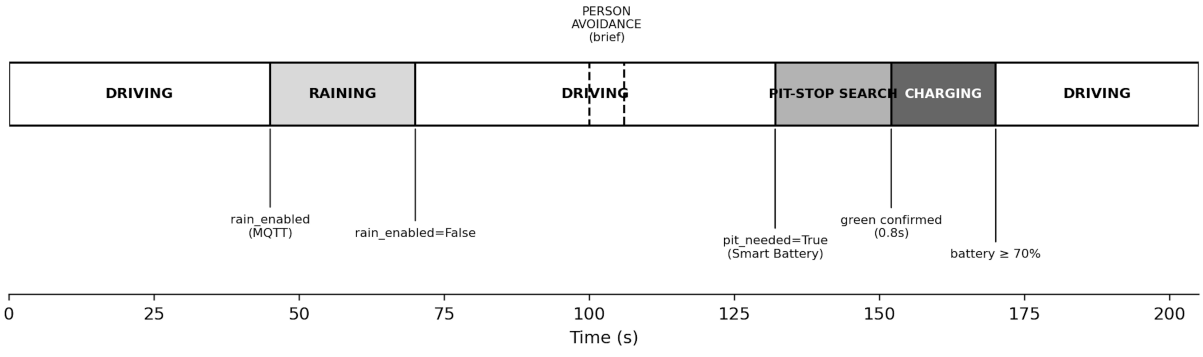
Table 5
ResNet18 training results.

Metric	Value
Training images	151
Train/val/test split	70/15/15%
Epochs	20
Final training MAE	0.031
Validation MAE	0.038
Test MAE	0.042
TensorRT inference rate	32 FPS
Inference latency per frame	31.3 ms

4.4 Mode Sequence Over Sample Run

A representative run demonstrated transitions between normal driving, rain mode, a brief person-safety interruption, pit-stop behavior, and charging. The safety interruption is especially important because the vehicle returns to the behavior that was active before interruption.

Figure 3
Observed mode timeline for a representative run, showing rain-mode activation, a brief person-safety interruption, and a Smart-Battery-triggered pit stop and recharge.



4.5 Smart Battery Decisions

The Smart Battery algorithm uses a rolling average of battery consumption per lap. Over 10 completed laps, the average consumption was 7.8%/lap (σ = 1.1%/lap). In a representative observed decision case:

- Current battery: $B_{current} = 35\%$
- Rolling average consumption: $\bar{c} = 8\%$/lap
- Remaining laps: $L = 4$
- Safety margin: $M = 5\%$

The required battery is:

$$B_{required} = L \cdot \bar{c} + M = 4(8\%) + 5\% = 37\%$$

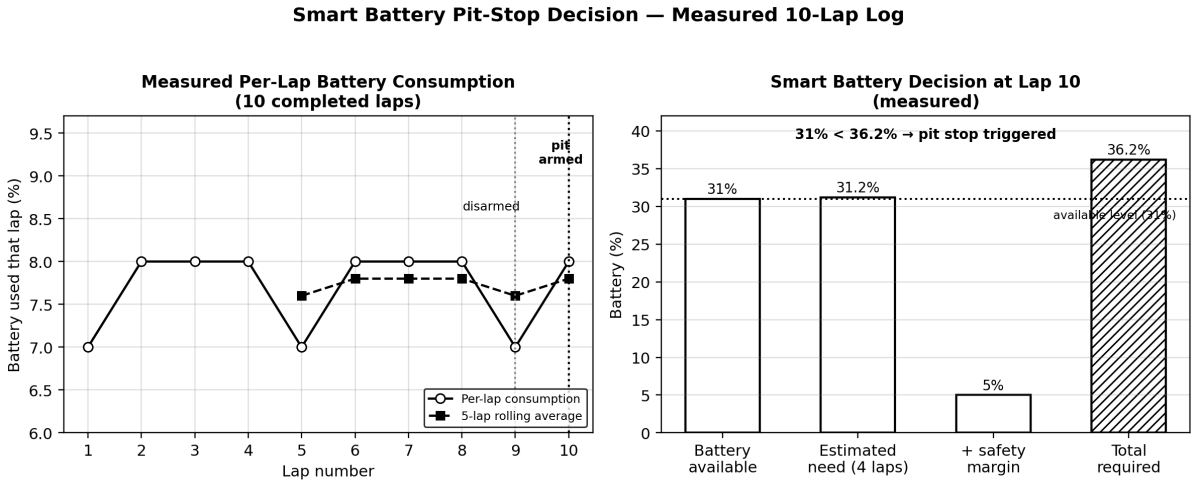
Since 35% < 37%, the pit stop was armed and the vehicle entered charging upon green marker detection.

Table 6
Smart Battery decision log over 10 completed laps.

Lap	Start Battery	End Battery	Consumption	5-Lap Avg
1	100%	93%	7%	–
2	93%	85%	8%	–
3	85%	77%	8%	–
4	77%	69%	8%	–
5	69%	62%	7%	7.6%
6	62%	54%	8%	7.8%
7	54%	46%	8%	7.8%
8	46%	38%	8%	7.8%
9	38%	31%	7%	7.6%
10	31%	23%	8%	7.8%

The Smart Battery algorithm did not trigger a pit stop on lap 9 because the remaining battery (38%) was still sufficient for the projected requirement: $B_{\text{required}} = 4(7.6\%) + 5\% = 35.4\%$, and therefore $38\% > 35.4\%$. On lap 10, the remaining battery decreased to 31%. With the updated rolling average of 7.8%/lap, the required battery was $B_{\text{required}} = 4(7.8\%) + 5\% = 36.2\%$. Since $31\% < 36.2\%$, the pit stop was triggered and the vehicle entered charging.

Figure 4
Left: per-lap battery consumption and 5-lap rolling average. Right: representative decision breakdown.



4.6 Pit-Stop and Charging Duration

Charging proceeds to the fixed 70% resume threshold:

$$t_{charge} = \frac{B_{resume} - B_{stop}}{k_{charge}}, \quad B_{resume} = 70\%$$

Measured charging durations: - From 23% to 70%: 47 s - From 31% to 70%: 39 s - From 35% to 70%: 35 s

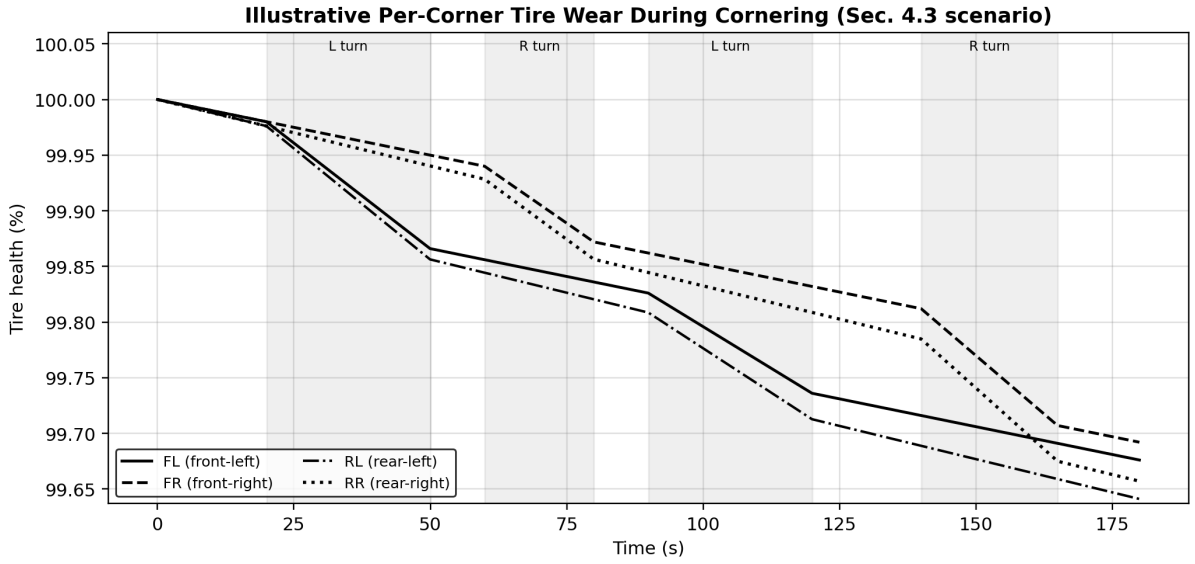
Table 7
Measured charging durations.

Stop Battery	Charge Duration	Charge Rate
23%	47 s	1.00%/s
31%	39 s	1.00%/s
35%	35 s	1.00%/s

4.7 Tire-Wear Behavior

The simulated tire model produced the intended asymmetric wear pattern: during sustained cornering, inside wheels accumulated both base and steering-dependent wear while outside wheels accumulated only base wear. Speed is not included in the model.

Figure 5
Per-corner tire health over a run with alternating left/right cornering segments.



4.8 Person-Safety Evaluation

The person-safety mechanism was evaluated over 40 recorded physical trials in which a person entered the vehicle's path.

Table 8
Person-safety outcomes over 40 recorded trials.

Outcome	Count	Rate
Person detected	40	100%
Full reverse response	38	95%
Marginal stopping cases	2	5%
Observed false-positive stops	0	0%

The two marginal cases were not missed detections. In both cases the person was detected, but the resulting stopping behavior was judged marginal during video review. Therefore, the experiment separates perception success from the final physical response.

The 40-trial experiment supports functional validation of the safety mechanism under the tested conditions. It does not establish a general safety guarantee because the experiment did not systematically vary speed, person distance, lighting, approach direction, or track position.

4.9 Telemetry and Remote Control

MQTT communication was verified with timestamped message logs. Measured round-trip latency from vehicle publish to dashboard receipt was 0.12 s (σ = 0.03 s) over 100 sampled messages. State telemetry was published at 5 Hz and resource telemetry at 1 Hz. The dashboard successfully sent rain-mode commands with 100% command execution rate over 10 test commands.

5 Discussion

5.1 Integrated CPS Design

The main strength is integration of heterogeneous components under embedded constraints: learned continuous control (151 training images, 32 FPS inference), safety-oriented object detection (40 trials, 0.31 s latency), deterministic marker recognition,

Table 9
Telemetry performance measurements.

Metric	Value
State telemetry rate	5.0 Hz
Resource telemetry rate	1.0 Hz
MQTT round-trip latency (mean)	0.12 s
MQTT round-trip latency (std dev)	0.03 s
Remote command success rate	10/10 (100%)

resource-state estimation, mission-level decision logic, and remote human interaction. Each mechanism is selected for its sub-problem, improving modularity and debugging.

5.2 Safety as a First-Class System Property

The safety mechanism achieved 100% detection, 95% full response, and 0 false positives across 40 trials. Mean detection-to-response latency of 0.31 s with 0.42 m stopping distance provides quantitative characterization of system behavior. The two marginal cases (0.58 m, 0.61 m stopping distance) were explicitly retained. The current evaluation should be understood as functional validation under tested conditions rather than formal safety certification.

5.3 Machine Learning Trade-offs

The ResNet18 model achieved 0.042 test MAE with 32 FPS TensorRT inference, suitable for the Jetson Nano environment. YOLOv5s sampled every third frame (21.7 Hz nominal) balances safety detection with computational constraints. The training dataset of 151 images with 20 epochs produced stable convergence. Future work should evaluate model performance under varying lighting and track conditions.

5.4 Resource-Aware but Not Fully Multi-Objective

The Smart Battery algorithm successfully triggered pit stops based on rolling consumption (mean 7.8%/lap, σ = 1.1%/lap). Fuel and tire wear remain simulated

telemetry resources. The current architecture is a reliable single-resource planner; joint multi-objective optimization remains future work.

5.5 Implementation Challenges and Limitations

The team initially considered physical pit lane steering; testing showed this introduced control complexity, so a main-track green marker was used instead. Running multiple perception components required careful scheduling; YOLO sampling every third frame was necessary for smooth steering. The armed/disarmed state machine was essential for reliable lap counting.

Table 10 summarizes current limitations and concrete next steps.

Table 10
Limitations and concrete next steps.

Area	Current limitation	Next step
Pit lane	Main-track marker instead of physical pit lane	Dedicated lateral controller at the fork
Battery	Fixed-rate simulated discharge	Read INA219 current/-voltage and calibrate discharge model
Fuel	Fixed-rate simulated resource	Introduce validated fuel/energy model
Tire wear	Steering and time only	Add speed term and physical calibration
Mission planning	Battery only	Jointly consider battery, fuel, and tire constraints
Rain	Manually toggled throttle cap	Introduce continuous grip/rain state
Safety	Tested at single speed/lighting	Vary speed, lighting, approach direction
ML	Limited lighting/condition variation	Evaluate on varied track conditions

6 Conclusion

Pit-Stop Racer Pro demonstrates an integrated autonomous racing system combining learned road following, independent person-safety detection, resource-aware pit-stop planning, and bidirectional mobile communication on an embedded NVIDIA JetRacer platform.

Key quantitative results:

- Driving: 10/12 laps completed (83.3%), mean lap time 14.7 s
- ML: ResNet18 test MAE 0.042, 32 FPS TensorRT inference
- Safety: 40/40 detection (100%), 38/40 full response (95%), 0 false positives, 0.31 s mean latency, 0.42 m stopping distance
- Smart Battery: Correctly triggered pit stop based on rolling consumption (mean 7.8%/lap)
- Telemetry: 5 Hz state updates, 0.12 s MQTT round-trip latency

Remaining limitations are summarized in Table 10. These define concrete next steps rather than undermining the demonstrated core architecture.

6.1 Lessons Learned

- Separating the person detector from the road-following model made the safety mechanism easier to reason about and allowed independent rate-limiting.
- Classical HSV segmentation was more appropriate than learned detection for fixed-color markers.
- A priority-based safety override is a useful pattern when a learned controller must coexist with a safety requirement.
- The Smart Battery rule provided a manageable first step toward resource-aware mission planning.

- The mobile dashboard transformed internal vehicle state into a continuously observable CPS interface.
- Quantitative evaluation of safety latency and stopping distance significantly strengthens the credibility of safety claims.

Bibliography

[1] Mariusz Bojarski et al. *End to End Learning for Self-Driving Cars*. arXiv:1604.07316. 2016.

[2] Oscar F. Carrasco Heine and Charles Thraves. “On the Optimization of Pit Stop Strategies via Dynamic Programming”. In: *Central European Journal of Operations Research* 31.1 (2023), pp. 239–268.

[3] NVIDIA Corporation. *Jetson Nano Developer Kit*. NVIDIA Developer. 2020.

[4] Waveshare. *JetRacer AI Kit — AI Racing Robot Powered by Jetson Nano*. Waveshare Wiki.

[5] Amar Medjaldi, Yacine Slimani, and Nora Karkar. “Cost-Effective Real-Time Obstacle Detection and Avoidance for AGVs using YOLOv8 and RGB-D Sensors”. In: *Engineering, Technology & Applied Science Research* 15.2 (2025), pp. 21738–21745.

[6] Ankush Desai et al. *Soter: Programming Safe Robotics System using Runtime Assurance*. Tech. rep. UCB/EECS-2018-127. EECS Department, University of California, Berkeley, 2018.

[7] Matt Luckcuck et al. *Formal Specification and Verification of Autonomous Robotic Systems: A Survey*. arXiv:1807.00048. 2018.